

KOGI PLATFORM

Portfolio System

System Design Document

v2.1.0

Independent Worker Operating System

Kogi Team · March 2026

1. Overview

1.1 Purpose

The Kogi Portfolio System is the central domain engine of the Kogi Platform — the single source of truth for every entity belonging to an independent worker's portfolio. It defines the full data model, lifecycle rules, distribution strategy, and computational models that underpin kogi-portfolio and every other application in the Kogi OS.

This document covers the complete design of portfolio_system.rs (v2.1.0) as it exists in the codebase and as it relates to the broader platform context defined in the Kogi System Design Document (SDD).

Design Principle: Everything in the Kogi ecosystem is a Portfolio Item. The Portfolio System is therefore the universal abstraction layer that every platform application builds upon.

1.2 Platform Context

The Portfolio System lives within KOGI-OS and feeds every downstream system:

Layer	System	Relationship to Portfolio System
Application	kogi-portfolio	Primary consumer — user-facing portfolio management UI
Application	kogi-project	All projects are Portfolio Items of type Project
Application	kogi-gig	Life management mapped to portfolio financial/time items
Application	kogi-marketplace	Assets and capabilities sourced from portfolio items
Application	kogi-community	Portfolio items publishable to community feeds
Engine	KOGI-ENGINE (AI)	Reads portfolio state for optimization, scoring, recommendations
Engine	KOGI-ENGINE (Data)	Portfolio events stream to Data Lake / Feature Store
Infra	KOGI-BASE	Persistence, backup, security enforced at infra layer

Layer	System	Relationship to Portfolio System
Infra	KOGI-MANAGER	RBAC and policy administration consume portfolio governance hooks

1.3 Key Design Goals

- Universal abstraction — every platform entity is representable as a Component (Item or Container)
- Event-sourced, auditable — every mutation appended to an immutable EventLog
- Distributed-first — VectorClock + CRDT log (LWW + OR-Set) for multi-node / federated operation
- Permission-gated — all mutations validated against a tiered PermissionTier hierarchy
- Policy-pluggable — governance PolicyEngine trait allows platform-level and custom rules
- Extensible — PortfolioPlugin trait + ToolBox integration (TMS) for runtime extension
- Computationally rich — 11 built-in analytical models covering health, risk, resource, and maturity

2. Structural Hierarchy

2.1 Top-Level Architecture

The system's primary structural tree is:



2.2 Component Taxonomy

A Component is the universal node. It is either an Item (leaf work entity) or a Container (organising structure).

2.2.1 Items

Item Type	Payload Struct	Key Fields
Portfolio	PortfolioItemPayload	mission, focus_areas, kpis: Vec<Metric>
Program	ProgramPayload	objective, projects: Vec<ComponentId>, budget
Project	ProjectPayload	project_type, methodology, sprints, backlog, releases

Item Type	Payload Struct	Key Fields
Resource	ResourcePayload	resource_type, skills, availability, hourly_rate
Artifact	ArtifactPayload	artifact_type, file_refs, produced_by
Asset	AssetPayload	asset_type, valuation, currency, acquired_at
SubPortfolio	SubPortfolioPayload	parent_portfolio_id, scope, strategic_goals

ProjectType variants: Organizational · Creative · Technical · Research · AI · Software · Media · Marketing · Investment · ContentCreator · DIY · Custom

AssetType variants: IntellectualProperty · Financial · Physical · Digital · Creative · Custom

ResourceType variants: Human · Machine · Service · License · Custom

2.2.2 Containers

Container	Sub-type / Kind	Purpose
Binder	—	Logical collection of items; carries dashboards
Book	Notebook	Free-form note pages
Book	ContactBook	Contacts + contact groups
Book	PlayBook	Triggers, plays, play steps
Book	ScheduleBook	Schedules + time blocks
Book	PlanBook	Goals, strategies, initiatives
Book	GuideBook	Versioned documentation sections
Book	ItemBook	Rich per-item dossier (dashboard, charter, workspace, catalogue, library, templates, logs, metrics, version history, schedule, directory)
Record	—	Schematised key-value records linked to a component
Folder	—	File-system-like hierarchy with FileRefs

Container	Sub-type / Kind	Purpose
Registry	—	Typed key-value registry with searchable entries
Archive	—	Deep storage; encrypted, compressible, retention-policy governed

3. Data Model

3.1 ComponentMetadata

Attached to every component; the universal identity and provenance block.

Field	Type	Description
id	Uuid (ComponentId)	Globally unique identifier
owners	Vec<UserId>	Primary owner list
tags	HashSet<String>	Flat string tag set
policy_ids	Vec<PolicyId>	Governance policies attached
created_at / updated_at	DateTime<Utc>	ISO-8601 timestamps
last_actor	String	Node ID of last writer (CRDT tiebreak)
vector_clock	VectorClock	Logical clock per node for causal ordering
properties	HashMap<String, JSON>	Arbitrary extensible metadata
version	VersionString	Semver string — managed via bump_version()
budget / budget_spent	f64	Tracks resource allocation inline
resource_units	f64	Generic resource unit counter

3.2 ComponentData

The shared mutable body of every component node.

Field	Type / Values	Description
category	ComponentCategory	Item(ItemCategory) Container(ContainerCategory)
name / description	String	Display identity
status	ComponentStatus	Draft · Active · Paused · Completed · Archived ·

Field	Type / Values	Description
		Deleted · Deprecated · UnderReview · Rejected · Custom
state	ComponentState	Initializing · Configured · Running · Idle · Blocked · Failing · Recovering · Migrating · Locked · Sealed · Custom
visibility	Visibility	Private · Protected · Public · Unlisted · DraftOnly
children / parents	Vec<ComponentId>	DAG parent-child relationships
links / dependents / dependencies	Vec<ComponentId>	Arbitrary link graph + dependency tracking
users	ComponentUsers	Tiered user registry (see §3.3)
analytics	ComponentAnalytics	Engagement counters (see §3.4)
risks	Vec<Risk>	Risk records with severity + mitigation
hashtags / topics	HashSet<String>	Social discovery signals
toolbox_ids	Vec<ToolBoxId>	TMS ToolBox references (v2.1 addition)
plugin_configs	HashMap<String, JSON>	Per-plugin configuration store

3.3 ComponentUsers — Permission Tier Hierarchy

All permission checks resolve against PermissionTier, which forms a total order. Every action requires a minimum tier.

Tier	Ordinal	Granted Capabilities
Viewer	0	Read-only, anonymous access
Subscriber	1	Bookmark, save, follow, subscribe
Contributor	2	Comment, react, poll, join

Tier	Ordinal	Granted Capabilities
Editor	3	Edit content, bump version
Manager	4	Manage settings and members
Owner	5	Full ownership privileges, all actions
Admin	6	Super-admin / platform-level override

ComponentUsers additionally tracks: watchers, subscribers, followers, investors, donors — each as a `HashSet<UserId>` — enabling fine-grained social graph queries without requiring a graph database for common lookups.

3.4 ComponentAnalytics

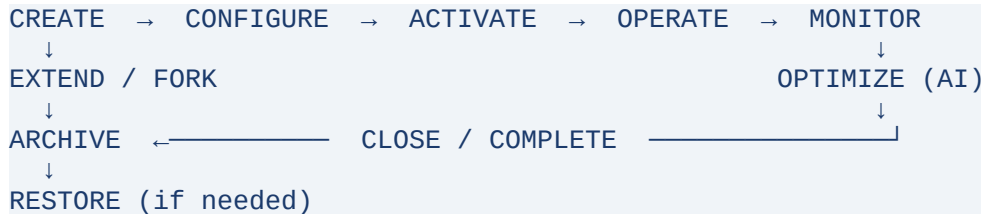
In-process engagement counters updated by `apply_action()` on every `ActionKind`. These feed the platform analytics pipeline:

Counter	Driven by ActionKind	Platform Metric Mapped
views	View	Impressions / reach
likes	Like	Approval signal
comments	Comment	Depth of engagement
shares	Share	Virality / brand advocacy
followers	Follow / Unfollow	Audience growth rate
saves	Save / Bookmark	Deferred intent / content value
click_through_rate	Click	CTR — links within component
engagement_rate	Computed	Total engagements ÷ reach
spread	Hashtag, Tag, Label	Active hashtag / tag count across platform

4. Lifecycle Management

4.1 Component Lifecycle States

Status and State are orthogonal axes. Status describes the high-level lifecycle phase; State describes the fine-grained operational condition.



4.2 Status Transitions (select)

From Status	Action	To Status	Side Effect
Draft	Post { visibility }	Active	Visibility set
Active	Archive	Archived	State → Sealed
Archived	Restore	Active	State → Configured
Any	Delete (soft)	Deleted	Edges retained
Any	Delete (hard)	Deleted	Edges purged, children orphaned
Any	bump_version()	—	Version string incremented, VersionHistory entry appended

4.3 Action System

Every user interaction is an ActionKind enum variant. `apply_action()` validates permissions, updates analytics, appends an ActivityLog entry, touches the VectorClock, and executes side effects.

Action categories:

- Social: Like, Comment, Share, Follow, Unfollow, Subscribe, Bookmark, Save, Watch, Invite, Poll
- Content: Post { visibility }, Edit, Create, Delete, Tag, Label, Hashtag, Mention
- Economic: Donate { amount }, Invest { amount, equity }
- Access: Join, Leave, Own { tier }
- System: Archive, Restore, Report, Search, Filter, Index, Notify, Alert

5. Graph & Edge Model

5.1 GraphEdge

All relationships between components — and between federated portfolio systems — are GraphEdge records. They are stored as a flat `Vec<GraphEdge>` on `PortfolioSystem` and participate in CRDT sync.

Edge Kind	Direction	Semantics
Hierarchy	Parent → Child	Compositional containment (Portfolio → Program → Project)
Dependency	A depends on B	Causal / blocking relationship; cycle detection enforced
Link	A ↔ B	Soft association — informational cross-reference
Contains	Container → Item	Container membership (Binder holds Items)
Federation	Cross-system	Cross-portfolio link (PortfolioFederation.federate())

5.2 Cycle Detection

`add_dependency()` performs shallow cycle detection: it rejects a dependency if the target is already listed as a dependent of the source. Full deep-cycle detection is delegated to the system-level graph traversal (`detect_cycle()` via DFS with a visited set).

5.3 Structural Primitives

Beyond the Component graph, five lightweight primitives allow flexible grouping without creating heavyweight Component nodes:

Primitive	Ordered?	Purpose
Group	No	Linked peer set — bidirectional; members cross-reference each other
Collection	No	Unordered set of component IDs — arbitrary membership

Primitive	Ordered?	Purpose
List	Yes	Ordered sequence — append, insert, move, remove
Schedule	Causal	Time-ordered list of items with event timestamps
Directory	Spatial	Path-based spatial map (path string → component IDs)

6. Distributed Architecture & CRDT

6.1 VectorClock

Every ComponentMetadata carries a VectorClock (node_id → logical timestamp map). Mutations call clock.tick(node_id) before writing. Concurrent writes (concurrent_with()) are resolved by last-writer-wins using the last_actor string as a tiebreak.

Method	Behaviour
tick(node_id)	Increment this node's counter; return new value
merge(other)	Element-wise max of all counters
happened_before(other)	True iff self $\leq$ other on all nodes and $\neq$ other
concurrent_with(other)	True iff neither dominates the other

6.2 CrdtLog — Operation Types

CrdtOperation	Semantics
SetField { component_id, field, value, timestamp, actor }	LWW field update — accepted only if remote timestamp is newer or actor ID sorts higher
AddEdge { edge, timestamp, actor }	OR-Set add — idempotent edge insertion
RemoveEdge { edge_id, timestamp, actor }	OR-Set remove — removes by ID
AddToSet { component_id, set_name, value, tag, timestamp, actor }	OR-Set member add with unique tag
RemoveFromSet { component_id, set_name, tag, timestamp, actor }	OR-Set member remove by tag

6.3 Federation

PortfolioFederation manages multiple PortfolioSystem instances (typically one per node or organisation). sync_crdt(source, target) pushes the source's CrdtLog into the target, triggering apply_crdt_log() which merges operations and fires all registered plugins.

FederationPeer records track endpoint URL, last-seen timestamp, trusted flag, and each peer's VectorClock — enabling the platform to detect stale or disconnected nodes.

7. Governance, Policy & Resource Management

7.1 PolicyEngine Trait

Any governance rule can be injected as an `Arc<dyn PolicyEngine>`. Engines are evaluated in registration order; the first non-Allow result is returned.

```
trait PolicyEngine: Send + Sync {  
  fn id(&self) -> &str;  
  fn name(&self) -> &str;  
  fn evaluate(&self, ctx: &PolicyContext<'_>) -> PolicyDecision;  
}  
  
enum PolicyDecision {  
  Allow,  
  Deny(String),  
  RequireApproval(String),  
}
```

`PolicyContext` exposes the full `Component`, the triggering `PortfolioEventKind`, the actor node ID, and a freeform properties map — giving engines all context needed to encode complex rules (budget caps, compliance flags, workflow gates).

7.2 Approval Workflow

When a policy returns `RequireApproval`, the caller can surface this to an approval workflow:

Step	Method	Notes
1. Request	<code>request_approval(component_id, reason)</code>	Creates <code>ApprovalRequest</code> (status: Pending), emits <code>ApprovalRequested</code> event
2. Resolve	<code>resolve_approval(request_id, approved, resolver, notes)</code>	Sets Approved or Rejected, emits appropriate event

7.3 Resource Allocation

`ResourceAllocation` tracks budget/resource consumption per component. `ResourceKind` covers: Budget, PersonHours, StoryPoints, ComputeUnits, StorageGiB, and Custom variants.

Method	Description
<code>allocate_resource(id, kind, total, denomination, period)</code>	Creates <code>ResourceAllocation</code> ; sets <code>metadata.budget</code>
<code>allocate_budget(id, total, currency, period)</code>	Convenience wrapper → <code>allocate_resource</code>

Method	Description
	with ResourceKind::Budget
record_consumption(id, amount)	Increments consumed; returns remaining; updates metadata.budget_spent
get_resource_allocation(id)	Read allocation for a component
overrun_allocations()	Returns all allocations where consumed > total

8. Computational Models

The Portfolio System ships 11 built-in analytical models. Each is a stateless compute struct with a `compute()` method, making them composable and testable in isolation.

8.1 Model Summary

Model	Primary Inputs	Output Score / Result
PortfolioHealth	Active ratio, budget utilisation, component count, avg risk severity	health_score 0–100
ProjectMetrics	Sprint velocity, completion %, backlog size, overdue count	completion_pct, velocity, risk_level
ProgramAlignment	Child project states, budget, KPI achievement rate	alignment_score 0–100
SubPortfolioRollup	Children components, resource allocations	rollup_score 0–100
ResourceUtilisation	Total capacity, committed, consumed, assigned items	utilisation_pct, demand_ratio, over_committed
AssetValue	Initial value, annual depreciation rate, age, cost basis	current_value, depreciation, roi_pct
ArtifactMaturity	Completeness, freshness (days), reuse count, reviewed flag	maturity_score 0–100
BinderCoverage	Expected IDs (set), actual IDs (set)	coverage_pct, missing_count, extra_count
BookConsistency	Section count, empty sections, schema compliance count	consistency_score 0–100
FolderOrganisation	Max depth, orphan count, duplicate names	organisation_score 0–100
RecordIntegrity	Entry count, duplicate count, integrity hash presence	integrity_score 0–100

8.2 PortfolioHealth Scoring Algorithm

The primary health signal exposed to the AI Engine and dashboard:

```

active_ratio    = active_components / total_components
budget_health   = clamp(1.0 - (consumed/allocated - 1.0).max(0.0), 0.0,
1.0)
risk_penalty    = avg_severity_score × 5.0    // Critical=4, High=3,
Medium=2, Low=1
size_bonus      = ln(1 + total_components) × 2.0

health_score    = clamp(
    active_ratio × 50.0
    + budget_health × 30.0
    - risk_penalty
    + size_bonus,
    0.0, 100.0
)

```

8.3 Integration with KOGI-ENGINE

Model outputs are not stored inside PortfolioSystem — they are computed on demand and streamed to the KOGI-ENGINE Data Platform via the event bus. The AI Engine then:

- Ingests health/risk scores into the Feature Store for ML model training
- Feeds scores to the Portfolio Agent for optimization recommendations
- Surfaces scores to dashboards via the Analytics Engine
- Triggers alerts if health_score drops below configurable thresholds

9. Event System & Audit Trail

9.1 EventLog

An append-only VecDeque<PortfolioEvent>. Every mutating operation on PortfolioSystem calls the internal emit() helper, which constructs a PortfolioEvent and notifies all plugins. The log is bounded (max 10,000 events) with FIFO eviction.

9.2 Event Kinds

Category	Event Kinds
Component	ComponentCreated · ComponentUpdated · ComponentRemoved · ComponentStatusChanged · ComponentStateChanged
Graph	EdgeAdded · EdgeRemoved
Governance	PolicyAttached · PolicyDetached · ApprovalRequested · ApprovalGranted · ApprovalRejected
Resource	ResourceAllocated · ResourceConsumed
Time-Travel	SnapshotSaved · CheckpointCreated · StateRestored
Distribution	CrdtMergeApplied
Action	ActionPerformed · UserAdded · UserRemoved

9.3 Snapshot & Time-Travel

Operation	Method	Description
Snapshot (transient)	snapshot()	Returns point-in-time view; not persisted
Snapshot (persisted)	save_snapshot(label)	Persisted to snapshots map; triggeres plugins
Checkpoint	save_checkpoint(label, note)	Named snapshot; immutable reference point
Restore	restore_snapshot(snapshot_id)	Replaces components + edges from snapshot; emits StateRestored

10. Tool Management System Integration (v2.1)

10.1 Overview

Version 2.1 introduced first-class integration with the Tool Management System (TMS). Any component — Item or Container — can now own one or more ToolBoxes. A ToolboxId is an opaque Uuid resolved at runtime via `ToolManagementSystem::get_toolbox(id)`.

Design Note: Toolbox references are stored on `ComponentData.toolbox_ids` (`Vec<ToolBoxId>`). The Portfolio System does not import or depend on TMS types directly — it holds only opaque UUIDs — preserving module independence.

10.2 TMS API on PortfolioSystem

Method	Permission Required	Behaviour
<code>attach_toolbox(user, component_id, toolbox_id)</code>	Owner	Idempotent — does not double-register same <code>ToolBoxId</code>
<code>detach_toolbox(user, component_id, toolbox_id)</code>	Owner	Removes <code>ToolBoxId</code> from component; no-op if not present
<code>has_toolbox(component_id, toolbox_id)</code>	Viewer	Returns <code>bool</code> — checks presence
<code>toolbox_ids(component_id)</code>	Viewer	Returns <code>Vec<ToolBoxId></code> for the component
<code>all_toolbox_ids()</code>	Viewer	Returns <code>HashSet<ToolBoxId></code> across all components in system

11. Plugin System

11.1 PortfolioPlugin Trait

Plugins are registered as `Arc<dyn PortfolioPlugin>` and receive lifecycle hooks for every significant system event. All hooks have default no-op implementations, so a plugin implements only what it needs.

Hook	Fired When
<code>on_component_created(component)</code>	<code>create_item()</code> or <code>create_container()</code> succeeds
<code>on_component_updated(component)</code>	<code>update_info()</code> , <code>act()</code> , or <code>bump_version()</code> mutates a component
<code>on_component_removed(component_id)</code>	<code>delete()</code> with <code>hard_delete=true</code>
<code>on_edge_added(edge)</code>	<code>attach_child()</code> , <code>add_dependency()</code> , or <code>add_link()</code>
<code>on_edge_removed(edge_id)</code>	<code>detach_child()</code> or <code>remove_dependency()</code>
<code>on_event(event)</code>	Every <code>emit()</code> call — all <code>PortfolioEventKinds</code>
<code>on_snapshot_saved(snapshot)</code>	<code>save_snapshot()</code> or <code>save_checkpoint()</code>
<code>on_crdt_merge(ops)</code>	<code>apply_crdt_log()</code> after a remote CRDT merge

11.2 Plugin Use Cases

- Analytics bridge: forward engagement events to the KOGI-ENGINE event stream
- Search indexing: re-index a component in the SearchEngine on every update
- AI feature extraction: compute embeddings when components are created or updated
- Notification dispatch: fire alerts via kogi-chat on status transitions
- Compliance audit: export events to immutable audit store for regulatory compliance

12. Search & Query

12.1 SearchQuery

Supports multi-dimensional filtering across the component store:

- text — full-text search across name and description
- tags / hashtags — set intersection filter
- categories — filter by ComponentCategory
- statuses — filter by one or more ComponentStatus values
- owner — filter by UserId
- visibility — filter by Visibility level
- created_after / created_before — time-range filter
- limit / offset — pagination

12.2 Portfolio Query Language (PQL)

PortfolioQuery provides a structured predicate model used by higher-level query APIs:

- categories, statuses, tags — typed set filters
- name_contains — case-insensitive substring match
- owner — single-owner filter
- has_dependencies / has_children — structural existence predicates
- property_filter — arbitrary (key, value) match against ComponentMetadata.properties

12.3 Search Result

SearchResult carries: component_id, name, category, status, score (relevance float), and matched_fields (Vec<String>) — enabling ranked result presentation and highlighting in the kogi-portfolio UI.

13. Error Model

All fallible operations return `PortfolioResult<T> = Result<T, PortfolioError>`. Error variants are designed to be machine-readable for API surfaces and UI localisation:

Error Variant	Payload	When Raised
NotFound	ComponentId	Component not registered in the system
PermissionDenied	user_id, action: String	Caller's PermissionTier is insufficient
InvalidOperation	message: String	Business rule violation (e.g. policy Deny)
CyclicDependency	ComponentId, ComponentId	Dependency would create a cycle in the graph
VersionConflict	local, remote: VersionString	Concurrent version mismatch on merge
AlreadyExists	ComponentId	Duplicate insert attempted
InvalidState	current: ComponentState, attempted: String	Action not valid for current operational state
StorageError	message: String	Persistence layer failure
ValidationError	message: String	Schema / field validation failure

14. ItemBook — Per-Item Rich Dossier

14.1 Overview

Every Item optionally carries an ItemBook (item_book: Option<ItemBookData>) — a living document that represents the full operational context of that item. When present, an ItemBook is also updated by bump_version() to maintain a VersionHistory.

14.2 ItemBookData Structure

Sub-component	Type	Contents
dashboard	Option<Dashboard>	Configurable widgets with layout map; powers in-app analytics views
charter	Option<Charter>	Executive summary, objectives, scope, stakeholders, success criteria, constraints, assumptions, risks, approval trail
workspace	Option<Workspace>	Active files, documents, content blocks, connected items, plugin references, rooms
catalogue	Option<Catalogue>	Tagged, searchable entries linking to child items / resources
library	Option<Library>	Reusable LibraryAssets: templates, plugins, workflows, snippets, schemas, playbooks
templates	Vec<Template>	Configuration and content templates for the item
logs	Vec<ActivityLog>	Item-scoped activity history
metrics	Vec<Metric>	Named numeric KPIs with unit and optional target
version	Option<VersionHistory>	Full semver history with author, message, diff_ref, and tags per entry
schedule	Option<ItemSchedule>	Events, milestones, recurring tasks, reminders

Sub-component	Type	Contents
directory	Option<Directory>	Path-based spatial organisation of linked components

14.3 Charter

The Charter is the governance core of an item. It captures all stakeholder-approved commitments and is itself versioned and approval-stamped. Risks within the Charter map directly to the Risk model (severity, probability, mitigation, owner, status).

15. API Surface Reference

15.1 PortfolioSystem — Core Methods

Method Group	Key Methods
Component CRUD	create_item() · create_container() · read() · read_mut() · update_info() · delete()
Graph	attach_child() · detach_child() · add_dependency() · add_link()
Actions	act(user, component_id, ActionKind)
Analytics	compute_portfolio_health() · compute_project_metrics() · compute_program_alignment() · compute_sub_portfolio_rollup()
Version Control	bump_version(actor, id, VersionPart, message)
Resource	allocate_resource() · allocate_budget() · record_consumption() · record_spend() · overrun_allocations()
Governance	register_policy_engine() · attach_policy() · detach_policy() · request_approval() · resolve_approval()
Snapshots	snapshot() · save_snapshot() · save_checkpoint() · restore_snapshot()
CRDT	apply_crdt_log() · crdt_log()
TMS	attach_toolbox() · detach_toolbox() · has_toolbox() · toolbox_ids() · all_toolbox_ids()
Plugins	register_plugin() · unregister_plugin()
Search / Query	search() · query()
Structural Primitives	create_group() · create_collection() · create_list() · create_schedule() · create_directory()

16. Platform Integration Map

16.1 Integration Points by Platform Application

kogi-* App	Portfolio System Dependency	Data Flow
kogi-portfolio	Primary consumer — full API	Read/write all component types; drives the user-facing portfolio management UI and ItemBook views
kogi-project	Items of type Project + ProjectPayload	Creates/reads Project items; reads Sprint, BacklogItem, Release payloads; pushes velocity to ProjectMetrics model
kogi-gig	Items of type Resource + Asset	Maps worker time and financial items to Portfolio resources; reads schedule from ScheduleBook
kogi-marketplace	Items of type Asset + visibility=Public	Reads published assets for listing; writes investment/donor interactions via ActionKind::Invest/Donate
kogi-community	ComponentData.visibility + Analytics	Reads public items for community feeds; analytics.spread drives hashtag trending
kogi-pay	ResourceAllocation (Budget kind)	Writes budget consumption via record_spend(); reads overrun_allocations() for alerts
kogi-chat	EventLog + plugin hooks	Plugin subscribes to on_event() to dispatch notifications and workroom messages
kogi-design	Items of type Artifact	Creates Artifact items from design outputs; links to parent Project via Hierarchy edges
kogi-dev	PortfolioPlugin trait + KOGI-SDK	Developers register custom plugins via register_plugin(); TMS ToolBoxes extend component capabilities

kogi-* App	Portfolio System Dependency	Data Flow
KOGI-ENGINE (AI)	Computational models + EventLog	Polls health/risk models; subscribes to event stream for feature extraction; writes recommendations back via ActionKind
KOGI-MANAGER	PolicyEngine + PermissionTier	Registers governance policies via register_policy_engine(); manages RBAC mapped to PermissionTier

17. Open Items & Future Considerations

17.1 Persistence Layer

PortfolioSystem is currently an in-memory data structure. The StorageError variant anticipates a persistence layer. The recommended path is a pluggable storage backend via a trait (e.g., PortfolioStore) with initial implementations for:

- PostgreSQL with JSONB for component data and edges
- Redis for hot-path reads and CRDT buffer
- Object store (S3-compatible) for snapshots and archive entries

17.2 Search Engine Integration

SearchQuery and PQL are defined but not backed by a full-text index. Production deployment should wire the on_component_created / on_component_updated plugin hooks to populate a dedicated SearchEngine (e.g., Meilisearch or Typesense) to support the kogi-portfolio UI search bar.

17.3 CRDT Conflict Resolution — Status Field

apply_crdt_op() currently skips status mutations (the "status" arm is a no-op). Status transitions carry business logic (e.g. state side effects) that LWW cannot safely handle. A merge strategy for status — possibly a custom CRDT lattice based on lifecycle order — should be specified before multi-node deployments.

17.4 Bounded EventLog

The EventLog is capped at 10,000 events with FIFO eviction. For long-running portfolios this will lose history. The recommended approach is to flush older events to the KOGI Data Lake (via a plugin) before eviction, ensuring audit continuity without unbounded memory growth.

17.5 TMS Deep Integration

v2.1 introduces opaque ToolBoxId references. The next milestone should define the runtime resolution protocol (how does a Portfolio System node call ToolManagementSystem::get_toolbox() without a hard dependency?) and specify how ToolBox lifecycle events (creation, deletion, version bumps) propagate back to components that reference them.

17.6 AI Agent Writeback Protocol

The AI Portfolio Agent produces optimization recommendations, risk scores, and strategic suggestions. A well-typed writeback protocol should be defined so the agent can create Metric entries, append governance_notes, update compliance_flags, and trigger ApprovalRequests — without bypassing permission checks.